



UNIVERSIDAD DE LAS FUERZAS ARMADAS “ESPE”

Departamento de Ciencias de la Computación

Carrera de Ingeniería de Software

Análisis y Diseño de Software

GUIA 02 PLAN UML

Estudiante:

Lucas Góngora

Klever Jami

Jordy Viscaino

Docente:

Ing. JENNY ALEXANDRA RUIZ ROBALINO

NRC: 30723

Fecha de Entrega:

05 de mayo de 2026

1. Propósito del Análisis

La presente guía tiene como finalidad analizar el CRUD de productos del sistema PaperBoost mediante la relación entre casos de uso, clases de análisis y diagramas de secuencia. A diferencia de un enfoque centrado en implementación, el propósito de esta etapa es identificar qué objetos participan en cada funcionalidad, cómo se distribuyen sus responsabilidades y de qué manera se mantiene la trazabilidad desde los requisitos funcionales del SRS hasta la interacción entre los elementos del modelo.

En este contexto, el análisis se concentra en las operaciones principales del módulo de inventario: registrar productos manualmente, consultar inventario, modificar datos de productos existentes y dar de baja productos del catálogo activo. Estos procesos representan el núcleo funcional del mantenimiento de inventario en PaperBoost y constituyen la base para derivar clases de análisis coherentes y secuencias alineadas con el comportamiento esperado del sistema.

2. Clases de Análisis

A partir de los casos de uso seleccionados en la guía anterior se identifican las clases de análisis necesarias para modelar la interacción del usuario con el módulo de inventario. Con el fin de mantener una adecuada separación de responsabilidades, se emplean las categorías frontera, control, entidad y repositorio, tal como se recomienda en el análisis orientado a objetos.

Las clases de frontera representan las pantallas o componentes visibles con los que interactúa el Administrador durante las operaciones del inventario. Las clases de control coordinan la lógica de cada caso de uso, validan condiciones y dirigen las operaciones sobre las entidades y repositorios. La entidad Producto representa la información principal del dominio, mientras que RepositorioProducto concentra las operaciones de almacenamiento, búsqueda, actualización, baja lógica y consulta del catálogo.

Tipo de clase	Responsabilidad en el análisis	Clase propuesta	Relación con el CRUD
Boundary / Frontera	Recibe acciones del actor y presenta resultados en la interfaz.	UI_Inventario, UI_FormularioProducto, UI_ConfirmacionBaja	Permiten consultar, registrar, editar y confirmar baja de productos.
Control	Coordina la lógica de cada caso de uso.	ControlConsultaInventario, ControlMantenimientoProducto	Gestionan consulta, validación, registro, actualización y baja lógica.
Entity / Entidad	Representa la información principal del dominio.	Producto	Contiene atributos del producto registrados en inventario.
Repository / Repositorio	Conserva y recupera registros sin entrar aún al diseño físico de base de datos.	RepositorioProducto	Permite guardar, buscar, actualizar, listar y cambiar estado del producto.

3. Diagrama de clases de Análisis

Código

```
@startuml
skinparam classAttributeIconSize 0
```

```
skinparam shadowing false
```

```
left to right direction
```

```
class "UI_Inventario" <<boundary>> {
    + solicitarInventario()
    + mostrarTabla(productos: List<Producto>)
    + mostrarDetalle(producto: Producto)
    + mostrarMensaje(mensaje: String)
}
```

```
class "UI_FormularioProducto" <<boundary>> {
    - txtSKU: String
    - txtNombre: String
    - txtPrecio: Double
    - txtStock: int
    - txtCategoria: String
    - txtUbicacion: String
    + capturarDatos()
    + enviarRegistro()
    + enviarActualizacion()
    + mostrarMensaje(mensaje: String)
}
```

```
class "UI_ConfirmacionBaja" <<boundary>> {
    + solicitarConfirmacion()
    + mostrarResultado(mensaje: String)
}
```

```
class "ControlConsultaInventario" <<control>> {
    + consultarInventario(): List<Producto>
    + buscarProducto(id: String): Producto
}
```

```
class "ControlMantenimientoProducto" <<control>> {
    + registrarProducto(datos: Producto): boolean
    + actualizarProducto(id: String, datos: Producto): boolean
    + darBajaProducto(id: String): boolean
    + validarDatos(datos: Producto): boolean
    + validarSkuUnico(sku: String): boolean
}
```

```
class "Producto" <<entity>> {
    - idProducto: String
    - sku: String
    - nombre: String
    - precioUnitario: Double
    - stockDisponible: int
}
```

```

- categoria: String
- ubicacion: String
- estado: String
+ Producto(idProducto: String, sku: String, nombre: String, precioUnitario: Double,
stockDisponible: int, categoria: String, ubicacion: String, estado: String)
+ actualizarDatos(sku: String, nombre: String, precioUnitario: Double, stockDisponible:
int, categoria: String, ubicacion: String): void
+ marcarInactivo(): void
}

```

```

class "RepositorioProducto" <<repository>> {
+ guardar(producto: Producto): void
+ buscarPorId(id: String): Producto
+ buscarPorSku(sku: String): Producto
+ actualizar(producto: Producto): void
+ listarTodos(): List<Producto>
+ darBaja(id: String): void
}

```

UI_Inventario --> ControlConsultaInventario : solicita consulta

UI_FormularioProducto --> ControlMantenimientoProducto : solicita registro/edición

UI_ConfirmacionBaja --> ControlMantenimientoProducto : confirma baja

ControlConsultaInventario --> RepositorioProducto : recupera datos

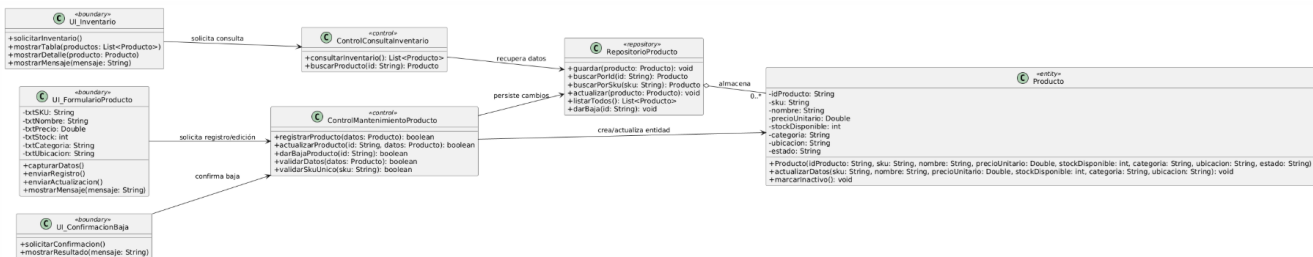
ControlMantenimientoProducto --> RepositorioProducto : persiste cambios

ControlMantenimientoProducto --> Producto : crea/actualiza entidad

RepositorioProducto o-- "0..*" Producto : almacena

@enduml

Resultado



Justificación

Este nuevo diagrama mejora la separación de responsabilidades y se alinea mejor con la derivación propuesta en la guía modelo, donde las clases deben corresponder a los casos de uso y a los mensajes que luego aparecerán en las secuencias.

Además, refleja con mayor fidelidad el dominio descrito por el SRS de PaperBoost, especialmente en los atributos del producto y en la distinción entre consulta, mantenimiento y baja lógica.

Secuencia de Análisis

Los diagramas de secuencia permiten representar cómo colaboran los objetos identificados en el diagrama de clases de análisis para cumplir cada caso de uso principal del CRUD de productos. Se elabora un diagrama por operación relevante para evitar sobrecarga visual y asegurar coherencia entre requisitos, clases y mensajes intercambiados durante la ejecución del flujo.

3.1 Secuencia Registro de Producto Manual (Crear / RQF-007)

Esta secuencia representa el flujo que ocurre cuando el Administrador registra manualmente un nuevo producto dentro del inventario. El proceso parte desde la captura de datos en la interfaz, continúa con la validación de campos obligatorios y unicidad del SKU, y concluye con la creación y almacenamiento del producto dentro del repositorio.

Código

```
@startuml
title Secuencia de análisis - Registrar producto manualmente
actor "Administrador" as Admin
boundary "UI_FormularioProducto" as UI
control "ControlMantenimientoProducto" as Ctrl
entity "Producto" as Prod
database "RepositorioProducto" as Repo

Admin -> UI : ingresa datos del producto
Admin -> UI : seleccionar Guardar
UI -> UI : capturarDatos()
UI -> Ctrl : registrarProducto(datos)

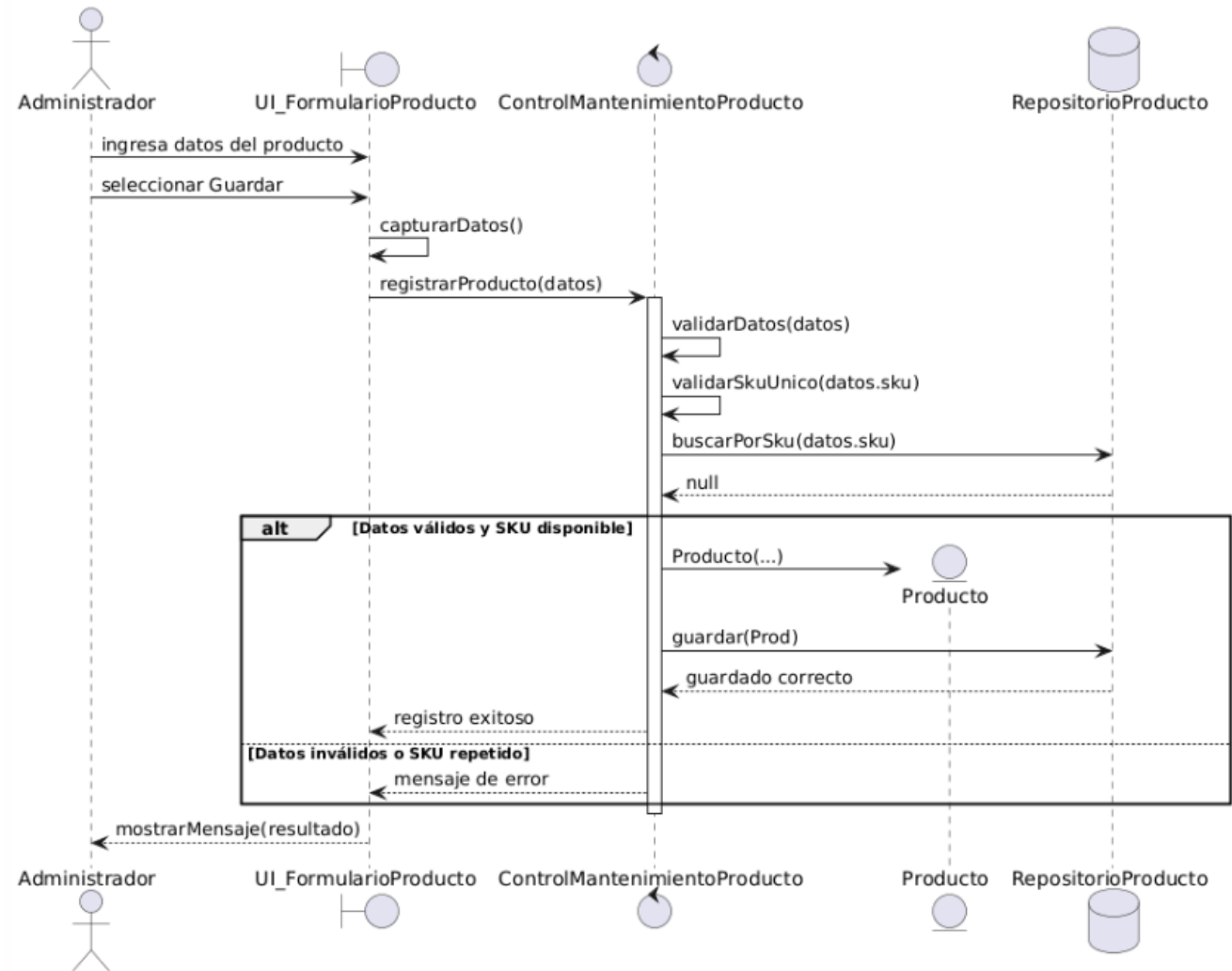
activate Ctrl
Ctrl -> Ctrl : validarDatos(datos)
Ctrl -> Ctrl : validarSkuUnico(datos.sku)
Ctrl -> Repo : buscarPorSku(datos.sku)
Repo --> Ctrl : null

alt Datos válidos y SKU disponible
  create Prod
  Ctrl -> Prod : Producto(...)
  Ctrl -> Repo : guardar(Prod)
  Repo --> Ctrl : guardado correcto
  Ctrl --> UI : registro exitoso
else Datos inválidos o SKU repetido
  Ctrl --> UI : mensaje de error
end

deactivate Ctrl
UI --> Admin : mostrarMensaje(resultado)
@enduml
```

Resultado

Secuencia de análisis - Registrar producto manualmente



Justificación

La secuencia de registro manual muestra cómo el sistema procesa el ingreso de un nuevo producto desde la captura de datos hasta su almacenamiento en el repositorio. El flujo refleja que la validación de campos y la verificación de unicidad del SKU son condiciones obligatorias antes de crear y guardar la entidad Producto.

Además, la secuencia permite evidenciar que la lógica del caso de uso se concentra en la clase de control, mientras que la interfaz solo captura información y comunica resultados al usuario. Esta distribución mantiene coherencia con el análisis orientado a objetos y con la trazabilidad definida para el módulo de inventario.

3.2 Secuencia: Consulta de Productos en Inventario (Leer / RQF-011)

Esta secuencia describe el comportamiento del sistema cuando el Administrador accede al inventario para visualizar el listado de productos registrados. El proceso consiste en solicitar los datos al control de consulta, recuperar la lista desde el repositorio y mostrarla en la interfaz con fines de revisión, búsqueda o selección.

Código

```

@startuml
title Secuencia de análisis - Consultar inventario
actor "Administrador" as Admin
    
```

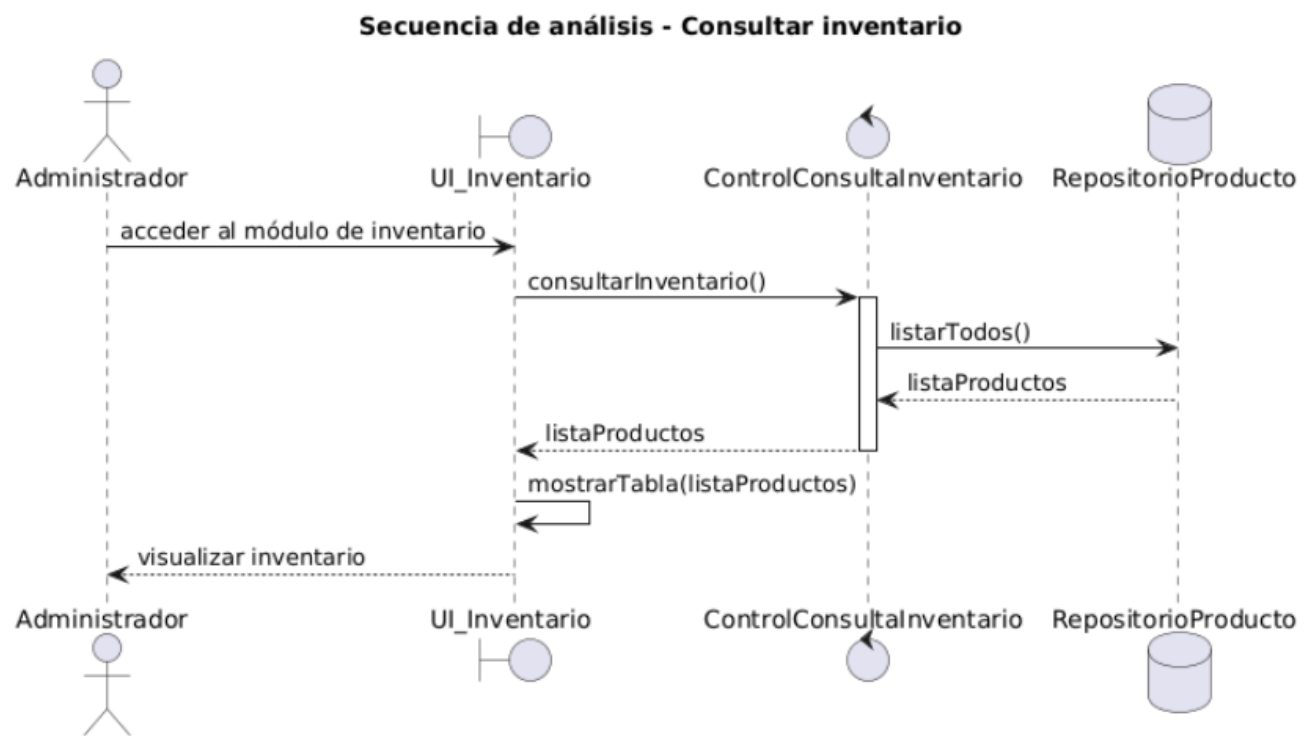
```

boundary "UI_Inventario" as UI
control "ControlConsultaInventario" as Ctrl
database "RepositorioProducto" as Repo
  
```

```

Admin -> UI : acceder al módulo de inventario
UI -> Ctrl : consultarInventario()
activate Ctrl
Ctrl -> Repo : listarTodos()
Repo --> Ctrl : listaProductos
Ctrl --> UI : listaProductos
deactivate Ctrl
UI -> UI : mostrarTabla(listaProductos)
UI --> Admin : visualizar inventario
@enduml
  
```

Resultado



Justificación

La secuencia de consulta representa la operación de lectura del inventario mediante un flujo simple y coherente con el comportamiento esperado del sistema. El Administrador solicita visualizar el inventario, la interfaz delega la acción al control y este recupera la lista de productos desde el repositorio para presentarla en pantalla.

Esta secuencia es importante porque refleja que la consulta constituye una operación independiente del mantenimiento de productos. De esta manera, el modelo diferencia correctamente la lógica de lectura frente a las operaciones de registro, actualización y baja.

3.3 Secuencia Editar Productos (Actualizar / RQF-013)

Esta secuencia representa el flujo seguido cuando el Administrador modifica la información de un producto existente dentro del inventario. El proceso incluye la recuperación del producto desde el repositorio, la validación de los nuevos datos, la actualización de la entidad y la persistencia final del cambio.

Codigo

```
@startuml
title Secuencia de análisis - Modificar datos de producto
actor "Administrador" as Admin
boundary "UI_FormularioProducto" as UI
control "ControlMantenimientoProducto" as Ctrl
entity "Producto" as Prod
database "RepositorioProducto" as Repo

Admin -> UI : seleccionar producto y editar datos
Admin -> UI : seleccionar Guardar cambios
UI -> UI : capturarDatos()
UI -> Ctrl : actualizarProducto(id, datos)

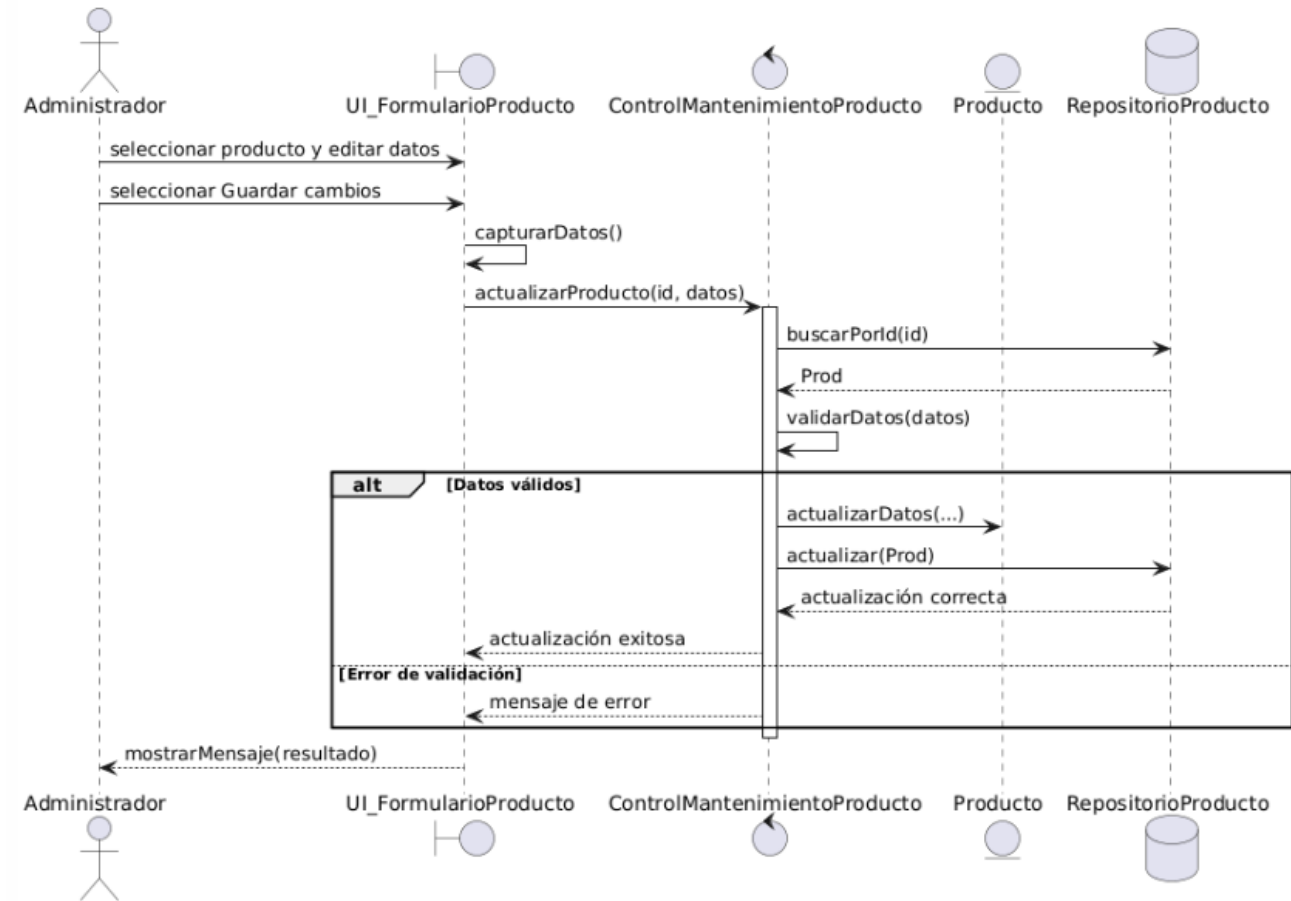
activate Ctrl
Ctrl -> Repo : buscarPorId(id)
Repo --> Ctrl : Prod
Ctrl -> Ctrl : validarDatos(datos)

alt Datos válidos
  Ctrl -> Prod : actualizarDatos(...)
  Ctrl -> Repo : actualizar(Prod)
  Repo --> Ctrl : actualización correcta
  Ctrl --> UI : actualización exitosa
else Error de validación
  Ctrl --> UI : mensaje de error
end

deactivate Ctrl
UI --> Admin : mostrarMensaje(resultado)
@enduml
```

Resultado

Secuencia de análisis - Modificar datos de producto



Justificación

La secuencia de edición describe cómo el sistema actualiza un producto existente a partir de nuevos datos ingresados por el Administrador. El control recupera el producto, valida la información y coordina la actualización de la entidad antes de persistir el cambio en el repositorio.

Este flujo demuestra que la modificación de datos no se realiza de forma directa, sino mediante un proceso controlado que asegura consistencia antes de guardar cambios. Así, la secuencia mantiene coherencia con el requisito funcional de edición y con la validación exigida por el sistema.

3.4 Secuencia: Eliminar / Dar de baja Producto (Borrar / RQF-023)

Esta secuencia modela el flujo mediante el cual el Administrador da de baja un producto del inventario, evitando que continúe disponible para nuevas transacciones. El proceso requiere una confirmación explícita, la verificación de existencia del producto y la ejecución de una baja lógica sobre el registro correspondiente.

Código

```

@startuml
title Secuencia de análisis - Dar de baja producto
actor "Administrador" as Admin
boundary "UI_ConfirmacionBaja" as UI
control "ControlMantenimientoProducto" as Ctrl
entity "Producto" as Prod
database "RepositorioProducto" as Repo
    
```

```

Admin -> UI : solicitar eliminación de producto
UI -> UI : solicitarConfirmacion()
Admin --> UI : confirmar acción
UI -> Ctrl : darBajaProducto(id)

activate Ctrl
Ctrl -> Repo : buscarPorId(id)
Repo --> Ctrl : Prod

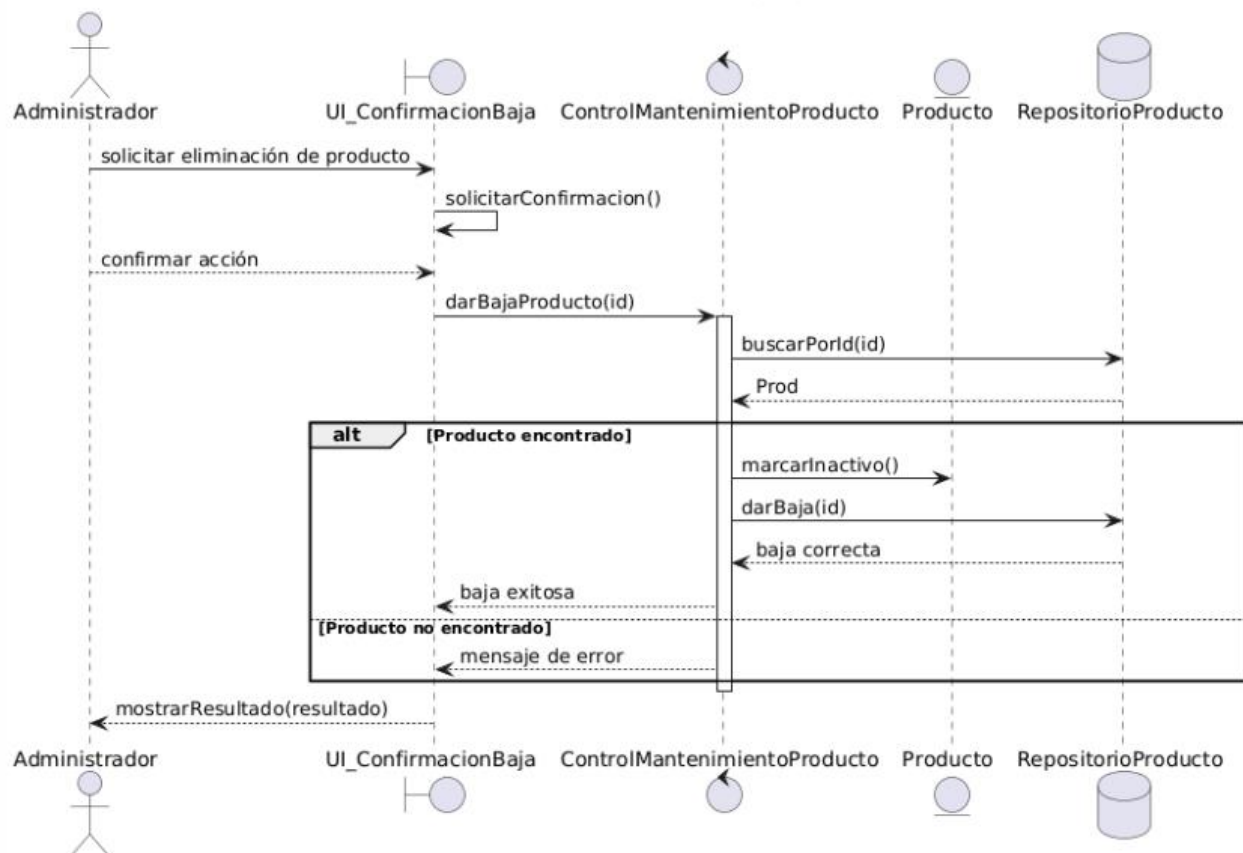
alt Producto encontrado
  Ctrl -> Prod : marcarInactivo()
  Ctrl -> Repo : darBaja(id)
  Repo --> Ctrl : baja correcta
  Ctrl --> UI : baja exitosa
else Producto no encontrado
  Ctrl --> UI : mensaje de error
end

deactivate Ctrl
UI --> Admin : mostrarResultado(resultado)
@enduml

```

Resultado

Secuencia de análisis - Dar de baja producto



Justificación

La secuencia de baja de producto representa un proceso controlado en el que el Administrador confirma la acción antes de ejecutarla. Luego de verificar la existencia del producto, el sistema actualiza su estado a inactivo y registra la baja en el repositorio.

Esta representación es adecuada porque el SRS define la eliminación como una baja lógica y no como un borrado físico del registro. De esta manera, el modelo conserva coherencia con el comportamiento esperado del inventario y con la trazabilidad del caso de uso correspondiente.

3.5. Casos de Uso

A continuación, se define el Caso de Uso (CU) correspondiente a cada Requisito Funcional (RF) establecido, detallando su propósito y relación para sentar las bases del modelado UML.

1. Requisito Funcional 007 (RQF-007)

"El sistema debe permitir registrar un nuevo producto ingresando manualmente sus datos (Nombre, SKU, Precio, Stock, etc.) en caso de no poseer o no utilizar código de barras."

Caso de Uso	CU-04: Registrar Producto Manualmente
Actor Principal	Administrador
Descripción	Permite al administrador ingresar los datos de un nuevo producto mediante un formulario para añadirlo al inventario.
Relaciones	<<include>> CU-Validar: Validar unicidad de SKU y campos obligatorios (RQF-008).

Código PlantUML (Diagrama Individual CU-04)

```
@startuml
left to right direction
skinparam packageStyle rectangle

actor "Administrador" as Admin

rectangle "Sistema PaperBoost" {
    usecase "Registrar Producto Manualmente" as UC1
    usecase "Validar Unicidad de SKU" as UC2
}
```

```
}

```

```
Admin -- UC1

```

```
UC1 ..> UC2 : <<include>>

```

```
@enduml

```



2. Requisito Funcional 011 (RQF-011)

"El sistema debe permitir consultar el inventario de productos mostrando la lista completa y resaltando aquellos con stock crítico."

Caso de Uso	CU-06: Consultar Inventario de Productos
Actor Principal	Administrador
Descripción	Permite al administrador visualizar el listado de todos los productos registrados en el sistema de manera rápida.
Relaciones	<<extend>> CU-Filtros: Aplicar filtros de búsqueda y ordenamiento (RQF-012).

Código PlantUML (Diagrama Individual CU-06)

```
@startuml

```

```
left to right direction

```

```
skinparam packageStyle rectangle

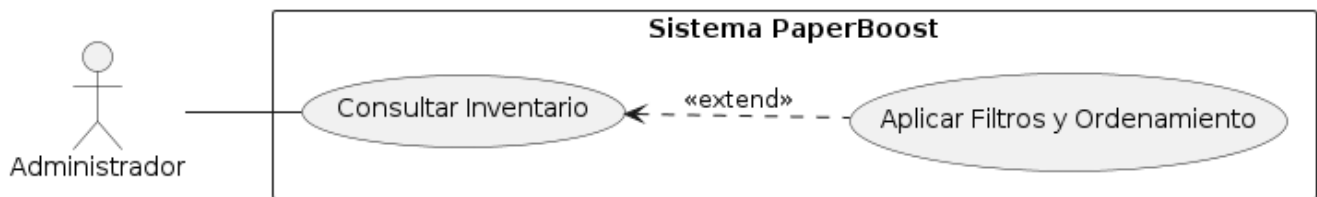
```

```

actor "Administrador" as Admin

rectangle "Sistema PaperBoost" {
    usecase "Consultar Inventario" as UC1
    usecase "Aplicar Filtros y Ordenamiento" as UC2
}

Admin -- UC1
UC1 <.. UC2 : <<extend>>
@enduml
    
```



3. Requisito Funcional 013 (RQF-013)

"El sistema debe permitir la edición o modificación de los datos de un producto previamente registrado en el catálogo."

Caso de Uso	CU-07: Modificar datos de producto existente
Actor Principal	Administrador
Descripción	Permite al usuario actualizar la información de un producto (como cambiar su precio o actualizar el nombre) seleccionándolo de la lista.
Relaciones	<<include>> CU-Validar: Validar los nuevos datos ingresados antes de guardar.

Código PlantUML (Diagrama Individual CU-07)

```

@startuml
left to right direction
skinparam packageStyle rectangle

actor "Administrador" as Admin

rectangle "Sistema PaperBoost" {
    usecase "Modificar Datos de Producto" as UC1
    usecase "Validar Nuevos Datos" as UC2
}

Admin -- UC1
UC1 ..> UC2 : <<include>>
@enduml
    
```



4. Requisito Funcional 023 (RQF-023)

"El sistema debe permitir eliminar o dar de baja un producto del inventario para que ya no esté disponible en futuras transacciones."

Caso de Uso	CU-12: Eliminar / Dar de baja producto
Actor Principal	Administrador
Descripción	Permite al administrador remover un producto del catálogo activo

	seleccionándolo y confirmando la acción.
Relaciones	<code><<include>></code> CU-Confirmar: Solicitar confirmación obligatoria de eliminación (RQF-024).

Código PlantUML (Diagrama Individual CU-12)

```

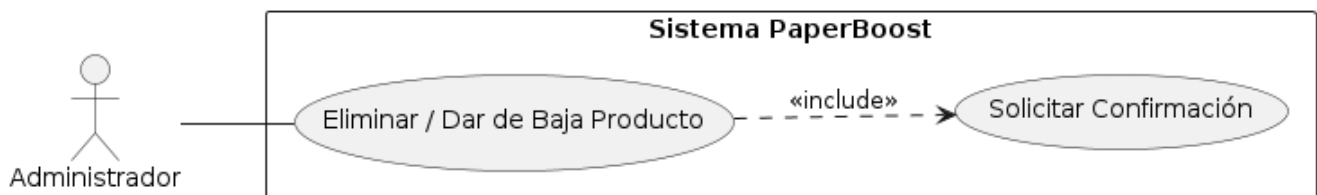
@startuml
left to right direction
skinparam packageStyle rectangle

actor "Administrador" as Admin

rectangle "Sistema PaperBoost" {
    usecase "Eliminar / Dar de Baja Producto" as UC1
    usecase "Solicitar Confirmación" as UC2
}

Admin -- UC1
UC1 ..> UC2 : <<include>>
@enduml

```



4. Matriz de trazabilidad RF-CU

Código RF	Requisito Funcional	Caso de Uso Asociado	Clases de Análisis Involucradas	Diagrama de Secuencia
RQF-007	Permitir registrar un nuevo producto manualmente.	Registrar Producto Manualmente (CUC-04)	UI_FormularioProducto, ControlMantenimientoProducto, Producto, RepositorioProducto	Secuencia: Registro de Producto Manual
RQF-011	Permitir consultar el inventario de productos.	Consultar Inventario de Productos (CUC-06)	UI_Inventario, ControlConsultaInventario, RepositorioProducto, Producto	Secuencia: Consulta de Productos en Inventario
RQF-013	Permitir la edición o modificación de un producto.	Modificar datos de producto existente (CUC-07)	UI_FormularioProducto, ControlMantenimientoProducto, Producto, RepositorioProducto	Secuencia: Editar Productos
RQF-023	Permitir eliminar o dar de baja un producto del inventario.	Eliminar Producto del Inventario (CUC-12)	UI_ConfirmaciónBaja, ControlMantenimientoProducto, Producto, RepositorioProducto	Secuencia: Eliminar / Dar de baja Producto
RQF-008	Validar campos obligatorios y unicidad de SKU.	Validar datos del producto	ControlMantenimientoProducto	Integrado en secuencias de Registro y Edición
RQF-024	Confirmación de baja y sincronización.	Confirmar baja de producto	UI_ConfirmaciónBaja, ControlMantenimientoProducto, RepositorioProducto	Integrado en secuencia de Eliminación

Justificación

La matriz de trazabilidad relaciona cada requisito funcional con su caso de uso, las clases de análisis involucradas y la secuencia correspondiente. Esto permite comprobar que todos los elementos del modelo tienen respaldo en el SRS y que el análisis mantiene coherencia entre sus diferentes artefactos.

Además, la matriz ayuda a identificar requisitos que actúan como comportamientos integrados dentro de otros flujos, como la validación de datos y la confirmación de baja. Por ello, se convierte en un elemento clave para demostrar consistencia y control dentro del análisis del sistema.

5. Conclusiones técnicas

- El modelo demuestra una trazabilidad entre todos los artefactos, donde cada requisito funcional se refleja consistentemente en un caso de uso, en métodos concretos dentro del controlador y en flujos explícitos dentro de los diagramas de secuencia. Esta alineación permite validar que la estructura estática (clases y métodos) responde exactamente al comportamiento dinámico esperado (interacciones en secuencia), evitando inconsistencias entre lo que el sistema debe hacer y cómo se implementa. Las secuencias cumplen un rol clave al verificar la correcta asignación de responsabilidades, evidenciando que la lógica reside en el controlador, la persistencia en el repositorio y la entidad se mantiene como un modelo de datos sin control de flujo. La matriz de trazabilidad consolida esta relación, permitiendo auditar el sistema, evaluar impacto de cambios y garantizar que no existan funcionalidades sin respaldo en requisitos.

